

Technische documentatie – Alarm en AI stem

1. Doel van mijn onderdeel

Mijn onderdeel van het project zorgt ervoor dat er een alarm afgaat wanneer de VisionAI detecteert dat de gebruiker wegstijgt. Het alarm bestaat uit 2 onderdelen:

1. Een alarm met geluid.
2. Een rood knipperende led lamp.

Het alarm wordt aangeroepen vanuit de VisionAI main.py via de functie:

Start_alarm()

```
if "looking away" in status and huidige_tijd - laatste_alarm_tijd > 10:
    start_alarm()
    laatste_alarm_tijd = huidige_tijd
```

2. Oude versie met Pico main.py

Eerst werkte het alarm via een Flask-website. In deze versie draaide er een lokale webserver op poort 5001.

De VisionAI kon de route /alarm aanroepen:

```
@app.route("/alarm")
```

```
def alarm():
```

Daarna werd de lamp gestart met:

```
asyncio.run(lamp_alarm())
```

Het geluid werd afgespeeld via javascript in de browser.

Probleem met deze versie

Deze oplossing werkte niet goed bij teamgenoten, omdat:

- De website lokaal moest draaien
- De juiste poort apart moet worden gerund

```
@app.route("/alarm")
def alarm():
    global alarm_active

    print("ALARM ROUTE AANGEROEPEN")

    alarm_active = True

    try:
        asyncio.run(lamp_alarm())
        print("Lamp alarm klaar")
        return "Alarm gestart"
    except Exception as e:
        print("Lamp fout:", e)
        return "Lamp fout", 500
```

- De browser open moet blijven

Daarom is de websiteversie vervangen door een gewone python-functie.

3. Nieuwe versie met alarm.py

De nieuwe oplossing gebruikt een los python bestand:

alarm.py

In dit bestand staan alle functies voor het alarm. Hierdoor hoeft er geen website of flask server meer worden gestart.

De functie start_alarm() start zowel het geluid als de lamp.

```
def start_alarm():  
    try:  
        geluid_alarm()  
        asyncio.run(lamp_alarm())  
        print("Alarm geactiveerd")  
    except Exception as e:  
        print("Alarm fout:", e)
```

4. Lampfunctie

De lamp wordt aangestuurd met de library:

Pywizlight

De lamp heeft op dit moment het ip-adres:

LAMP_IP = "172.20.10.14"

Dit kan veranderen als er een ander netwerk wordt gebruikt voor de lamp, maar dat betekent dat de lamp opnieuw moet worden ingesteld in de wiz app.

De functie lamp_alarm() maakt verbinding met de lamp:

lamp = wizlight(LAMP_IP)

```
async def lamp_alarm():  
    lamp = wizlight(LAMP_IP)
```

Daarna wordt de lamp eerst wit aangezet:

await lamp.turn_on(PilotBuilder(rgb=(255, 255), brightness=255))

```
await lamp.turn_on(PilotBuilder(rgb=(255, 255, 255), brightness=255))  
await asyncio.sleep(0.5)
```

Vervolgens knippert de lamp 5 keer rood:

```
for i in range(5):  
    await lamp.turn_on(PilotBuilder(rgb=(255, 0, 0), brightness=255))  
    await asyncio.sleep(0.3)  
  
    await lamp.turn_off()  
    await asyncio.sleep(0.2)
```

Aan het einde wordt de lamp weer wit gezet:

```
await lamp.turn_on(PilotBuilder(rgb=(255, 255, 255), brightness=255))
```

5. Geluidsfunctie

Het geluid wordt niet meer via de browser afgespeeld, maar direct vanuit Python.

Hiervoor wordt een alarmbestand gebruikt:

```
ALARM_WAV = "alarm_sound.wav"
```

De functie maak_alarm_geluid() maakt dit geluidsbestand automatisch aan als het nog niet bestaat.

Het geluid wisselt tussen 2 frequenties:

Namelijk 950 Hz en 1450 Hz

```
for i in range(12):  
    # 0.10 sec 950 Hz  
    for n in range(int(sample_rate * 0.10)):  
        value = volume if math.sin(2 * math.pi * 950 * n / sample_rate) > 0 else -volume  
        samples.append(value)  
  
    # 0.10 sec 1450 Hz  
    for n in range(int(sample_rate * 0.10)):  
        value = volume if math.sin(2 * math.pi * 1450 * n / sample_rate) > 0 else -volume  
        samples.append(value)
```

Hierdoor ontstaat een sireneachtig alarmgeluid. Vergelijkbaar met het geluid uit de oude websiteversie.

Het geluid wordt afgespeeld met:

```
geluid_alarm():  
    maak_alarm_geluid()  
    winsound.PlaySound(ALARM_WAV, winsound.SND_FILENAME | winsound.SND_ASYNC)
```

6. Hoofdfunctie van het alarm

De functie start_alarm() is de functie die VisionAI gebruikt.

```
def start_alarm():  
    try:  
        geluid_alarm()  
        asyncio.run(lamp_alarm())  
        print("Alarm geactiveerd")  
    except Exception as e:  
        print("Alarm fout:", e)
```

Deze functie doet 3 dingen:

1. Het geluid starten
2. Lampalarm starten
3. Foutmelding tonen als er iets misgaat

7. Toevoeging in VisionAI main.py

In VisionAI is bovenaan deze import toegevoegd:

From alarm import start_alarm

Hiermee kan de AI code de alarmfunctie gebruiken.

De oude websitekoppeling met requests is verwijderd + de websitekoppeling

Import requests, requests.get("http://127.0.0.1:5001/alarm", timeout=1)

Dit is vervangen door:

```
if "looking away" in status and huidige_tijd - laatste_alarm_tijd > alarm_tijd:  
    start_alarm()  
    laatste_alarm_tijd = huidige_tijd
```

8. Detectie van wegstijgen

VisionAI geeft bij het wegstijgen van de status:

“looking away”

Deze status wordt omgezet naar kleine letters:

```
status = str(analysis.get("status", "")).lower()
```

Daarna wordt er gecontroleerd of de gebruiker wegstijgt:

```
if "looking away" in status and huidige_tijd - laatste_alarm_tijd > alarm_cooldown:  
    start_alarm()  
    laatste_alarm_tijd = huidige_tijd
```

Als dit klopt wordt het alarm gestart.

9. Cooldown

Omdat VisionAI veel frames per seconde verwerkt, zou het alarm zonder beperking heel vaak achter elkaar starten.

Daarom is er een cooldown toegevoegd:

```
laatste_alarm_tijd = 0  
alarm_cooldown = 3
```

Daarna wordt gecontroleerd of er genoeg tijd tussen 2 alarmen zit:

```
if "looking away" in status and huidige_tijd - laatste_alarm_tijd > alarm_cooldown:  
    start_alarm()  
    laatste_alarm_tijd = huidige_tijd
```

Hierdoor kan het alarm maximaal 1 keer per 3 seconden afgaan.

10. De flow

De uiteindelijke gaat als volgt:

1. VisionAI detecteert “looking away”
2. Cooldown controleert of alarm opnieuw mag
3. Start_alarm() wordt gestart
4. Alarm gaat af
5. WIZ-lamp knippert rood
6. Lamp gaat terug naar wit

11. De AI-stem

Ook de AI-stem is onderdeel geworden van mijn bijdrage binnen het project. De AI-stem wordt gebruikt om de ervaring sterker naar te zetten en gebruikers direct het gevoel te geven dat zijn gecontroleerd en aangestuurd worden door een AI-systeem.

Ontwikkeling van de AI-stem

Voor het ontwikkelen van de AI-stem is gebruikgemaakt van een online AI voice generator.

Gebruikte tool: <https://quillbot.com/tools/ai-voice-generator>.

En de toon voor de AI: Maintain a robot, dark voice, which is forceful and mean.

Het doel hiervan was om een stem te genereren die past binnen het concept van het project.

Na het genereren is het audiobestand gedownload en toegevoegd aan het project in visual studio code.

Technische werking van de AI-stem

Voor het afspelen van de AI-stem wordt gebruikt gemaakt van pygame. Deze library maakt het mogelijk om geluidsbestanden binnen python af te spelen.

Introductiestem

Bij het starten van de ervaring wordt een introductiestem afgespeeld.

Code:

```
1  import pygame
2  import time
3
4  def speel_ai_intro():
5      pygame.mixer.init()
6      pygame.mixer.music.load("ai_intro.mp3")
7      pygame.mixer.music.play()
8
9      while pygame.mixer.music.get_busy():
10         time.sleep(0.1)
11
```

De werking hiervan gaat als volgt:

1. Pygame zoekt de audio.
2. Het audio bestand ai_intro.mp3 wordt geladen (dit is de audio voor de intro)
3. De AI-stem wordt afgespeeld waar de def functie staat in python bij de main.py.
4. Het systeem wacht totdat het geluidsbestand volledig is afgelopen.

Stem voor de taak 2

Naast de introductie is ook een 2^e stem toegevoegd.

Deze stem wordt gebruikt wanneer de gebruiker klaar is met het spel en wordt gestuurd naar taak 2.

Code:

```
import pygame
import time
import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

def speel_ai_taak2():
    pygame.mixer.init()

    pygame.mixer.music.load(
        os.path.join(BASE_DIR, "taak2.mp3")
    )

    pygame.mixer.music.play()

    while pygame.mixer.music.get_busy():
        time.sleep(0.1)
```

De werking hiervan gaat als volgt:

1. Het audiobestand taak2.mp3 wordt opgezocht.
2. Het bestand wordt geladen.
3. De AI-stem wordt afgespeeld.
4. Het systeem wacht tot het audiobestand klaar is.

Door deze code `os.path.join(BASE_DIR, "taak2.mp3")` kan het systeem het bestand vinden, ongeacht welke pc wordt gebruikt hiervoor.

De reden dat deze code verschillend is komt, doordat dit ipv een def functie in de main.py een functie moest worden in html, omdat het moet gaan starten wanneer het spel is afgelopen.

Daar is deze code voor bij gemaakt:

```
fetch('/speel-taak2-audio')
    .then(() => { console.log("taak 2 audio gestart"); })
    .catch(err => console.error("Audio fout:", err));
```

Doel van de AI-stem

De AI-stem ondersteunt meerdere onderdelen binnen het project.

- Het geeft meer beleving voor de gebruiker.
- De controle en prestatiedruk versterken.
- Het maakt de ervaring wat meer realistisch.
- Het geeft een gevoel aan de gebruikers dat het AI-systeem ze aanstuurt.